



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 03 — Hilos y concurrencia

Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



Lo que veremos en este bloque



El GIL

El candado de
CPython



CPU vs I/O

Cuándo aceleran
de verdad



Estados

El ciclo de vida
completo



Práctica

Trazas y el ring



Cierre

Errores y
entrevista

El GIL

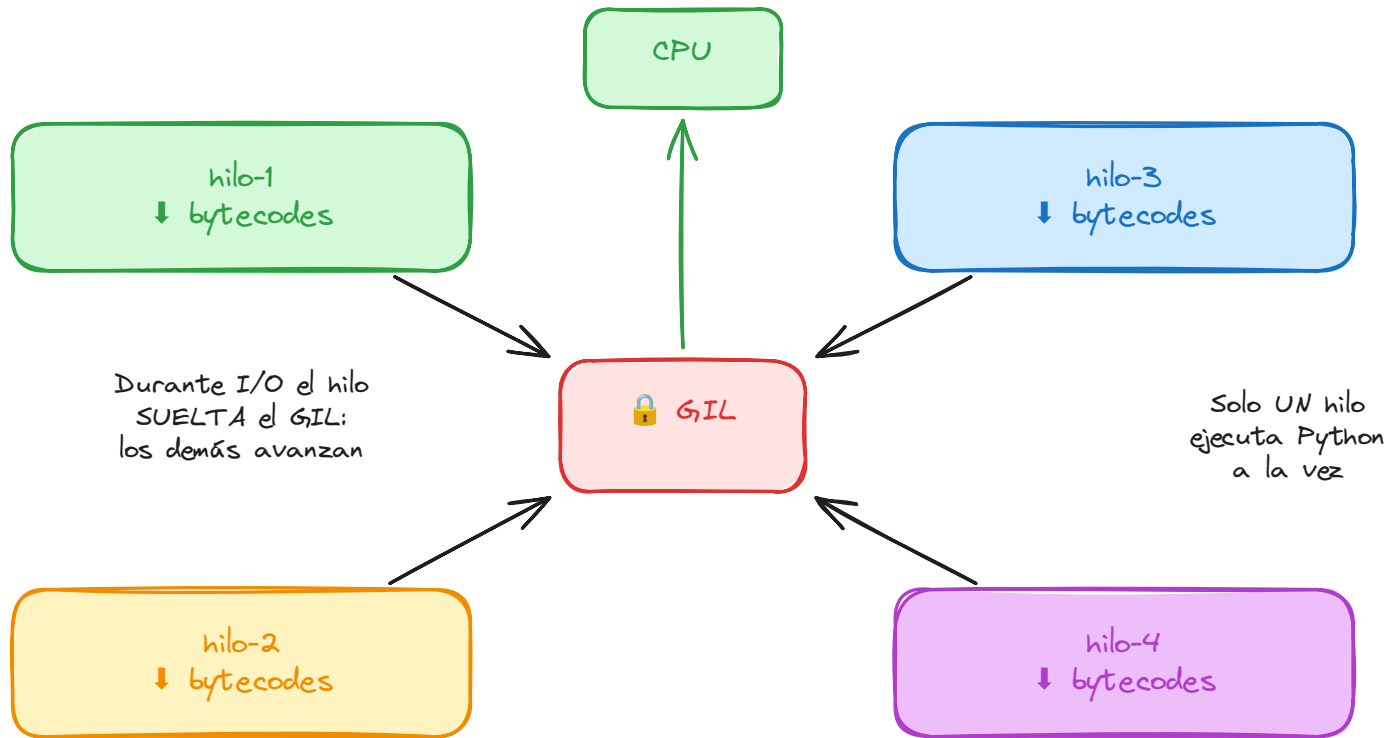
La idea en una frase

“

GIL = Global Interpreter Lock: un candado interno de CPython que evita que dos hilos ejecuten bytecode Python a la vez. Los hilos **no** aceleran el código de CPU... pero **sí** el de espera.

¿Por qué existe? Para proteger la memoria interna del intérprete: sin el candado, dos hilos podrían corromper las estructuras de Python al tocarlas a la vez. Python paga la seguridad con un límite: **no hay paralelismo real de CPU** entre hilos del mismo proceso.

El GIL en un dibujo



CPU-bound: los hilos NO sirven de nada

cpu_bound.py

Python

```
def contar():
    total = 0
    for i in range(50_000_000):
        total += i
inicio = time.time()
hilos = [threading.Thread(target=contar)
         for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"{time.time() - inicio:.2f}s")
```

Una tarea **CPU-bound** usa la CPU a tope: sumar millones de números, procesar imágenes, comprimir. El GIL no deja ejecutar a más de un hilo, así que:



4 hilos tardan lo mismo que 1 (a veces ligeramente más, por el coste de turnarse el candado).

I/O-bound: los hilos **SÍ** aceleran

io_bound.py

Python

```
def esperar():  
    time.sleep(2) # simula descarga  
inicio = time.time()  
hilos = [threading.Thread(target=esperar)  
         for _ in range(4)]  
for h in hilos: h.start()  
for h in hilos: h.join()  
print(f"{time.time() - inicio:.2f}s")  
# → ~2 segundos, no 8
```

Una tarea **I/O-bound** espera por algo externo: red, disco, una API. Al esperar, el hilo **libera el GIL** y los demás pueden trabajar.



Cuatro hilos que esperan 2 segundos **cada uno** terminan en **~2 segundos**: todos esperan *a la vez*.

La tabla que hay que saberse

Tipo de tarea	¿Hilos ayudan?	Motivo
CPU-bound (calcular, procesar)	 No	El GIL solo deja ejecutar a uno
I/O-bound (esperar red, disco)	 Sí	El GIL se libera durante la espera

 ¿Cómo distingo cada tipo? Si el programa gasta el tiempo **calculando** (contar, cifrar, procesar), es CPU-bound. Si lo gasta **esperando** (descargar, consultar una API, leer archivos), es I/O-bound. Los hilos brillan en el segundo caso.

Cuándo sirven de verdad los hilos



Red

10 descargas
con 10 hilos $\approx$
10× más rápido



Disco

Lecturas y
escrituras en
paralelo



Servidores

Cada cliente
espera; otros
avanzan (UD 6)



Cálculo puro

Hilos no: para
eso
multiprocessing

i Para CPU-bound en Python: `multiprocessing` lanza varios procesos, cada uno con su **propio GIL**: paralelismo real de CPU, al coste de crear procesos (UD 02).

Mini-chequeo: el GIL



Responde en 30 segundos:

1. ¿Qué significa exactamente que "CPython solo ejecuta un hilo a la vez"?
2. Descargar 5 archivos de 2 segundos cada uno: ¿1 hilo o 5? ¿Cuánto tarda cada opción?
3. Sumar 50 millones de números con 4 hilos: ¿más rápido, igual o más lento que con 1?



Respuestas:

- Que los hilos se **turnan** el bytecode en fragmentos pequeños: parecen simultáneos, pero nunca ejecutan Python a la vez
- **5 hilos:** ~2 segundos (los 5 esperan a la vez). Con 1 hilo: $5 \times 2 =$ **10 segundos**
- **Igual** (a veces peor): el GIL impide calcular en paralelo; para eso, multiprocessing

Estados del hilo

La idea en una frase

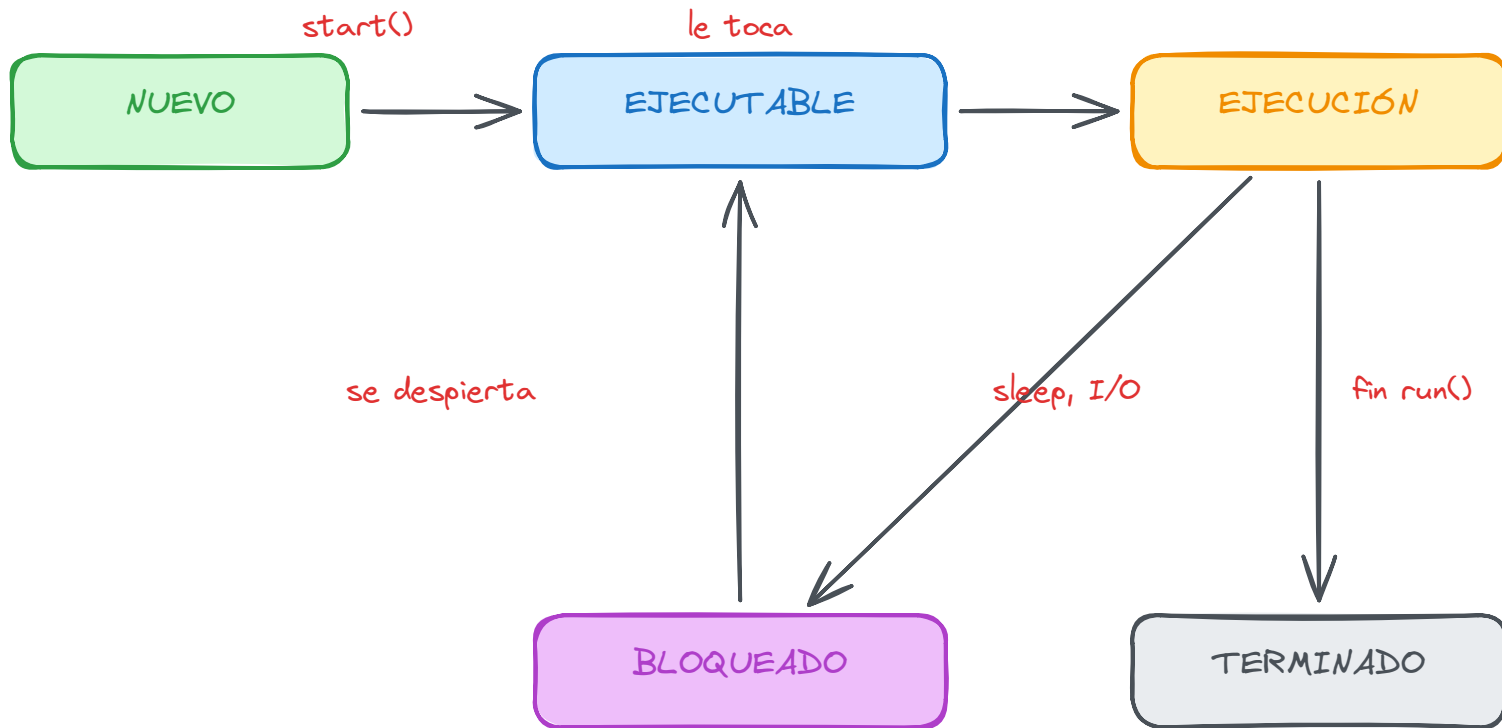
“

Un hilo nace como objeto, se lanza con `start()`, espera su turno, se ejecuta, se bloquea cuando espera algo y termina cuando su función acaba: ese es su **ciclo de vida**.

Saber en qué estado está un hilo (o cuándo lo estará) es la clave para entender por qué un programa multihilo se comporta como se comporta.

El ciclo de vida en un dibujo

CICLO DE VIDA DE UN HILO



Los estados, uno a uno

Estado	Significado
NUEVO	El objeto Thread existe pero no se ha llamado a <code>start()</code>
EJECUTABLE	<code>start()</code> llamado: puede ejecutar en cualquier momento
EJECUCIÓN	El scheduler le ha dado la CPU: ejecuta ahora mismo
BLOQUEADO	Esperando (sleep, I/O, un lock)
TERMINADO	Su función ha acabado; no volverá a ejecutarse

 De **BLOQUEADO** siempre se vuelve a **EJECUTABLE** (nunca directo a **EJECUCIÓN**): al despertar, toca esperar de nuevo su turno.

El estado BLOQUEADO, el más importante

En la práctica, el estado que más se ve es **BLOQUEADO**, casi siempre por:

- 1 `time.sleep(n)` — pide "no me des CPU durante n segundos"
- 2 Espera de **I/O** — descarga, lectura de archivo, un socket

✓ Mientras un hilo está BLOQUEADO **libera la CPU (y el GIL)**: otro hilo puede ejecutar. Es exactamente el mecanismo que hace rápidas las tareas I/O-bound.

i Curiosidad: `sleep(0)` cede la CPU **voluntariamente** sin esperar nada: buena práctica en hilos cooperativos.

¿Cuándo termina un hilo? `is_alive()`

is_alive.py

Python

```
def corto():  
    time.sleep(1)  
h = threading.Thread(target=corto)  
print(h.is_alive()) # False (NUEVO)  
h.start()  
print(h.is_alive()) # True  
h.join()  
print(h.is_alive()) # False (TERMINADO)
```

- 1 Un hilo llega a **TERMINADO** cuando su función acaba (o lanza una excepción no capturada)
- 2 Un **no daemon** terminado es requisito para que el programa pueda salir
- 3 Un **daemon** puede ser cortado en seco por el final del programa, sin llegar "bien" a **TERMINADO**

Mini-chequeo: estados



Responde sin mirar:

1. ¿En qué estado está un hilo recién creado, antes de `start()` ?
2. ¿Qué ocurre cuando un hilo en EJECUCIÓN llama a `time.sleep(2)` ?
3. ¿Puede un hilo pasar de BLOQUEADO a EJECUCIÓN sin pasar por EJECUTABLE?



Respuestas:

- **NUEVO:** el objeto existe, pero sin `start()` no ha ejecutado nada
- Pasa a **BLOQUEADO** 2 segundos: libera CPU y GIL; al despertar vuelve a EJECUTABLE
- **No:** siempre vuelve por EJECUTABLE y espera su turno del scheduler

Be the code, my friend

El hilo viajero

viajeros.py

Python

```
def viajero(nombre, paradas):  
    for i in range(paradas):  
        print(f"{nombre} parada {i+1}")  
        time.sleep(0.5)  
    print(f"{nombre} 🚩 destino")  
h1 = Thread(target=viajero, args=("Ana", 3))  
h2 = Thread(target=viajero, args=("Bob", 2))  
h1.start(); h2.start()  
h1.join(); h2.join()
```

Ana parada 1 · Bob parada 1
Bob 🚩 llegó a su destino
Ana parada 3
Ana 🚩 llegó a su destino



Ambos arrancan a la vez; mientras uno duerme (BLOQUEADO), el otro avanza. **Bob llega antes:** solo tiene 2 paradas. Y el orden exacto lo decide el scheduler: **no hay garantías.**

El ring: hilo vs proceso

¿Quién gana en cada asalto?



Hilo

- Ligero y barato: nace en milisegundos
- Comunica al instante con variables compartidas
- Brilla **esperando**: red, disco, clientes



Proceso

- Memoria aislada: un error no contagia a nadie
- Comunicación lenta: pipes, sockets, archivos
- Paralelismo **real de CPU**: cada uno con su GIL



Moraleja: procesos para aislamiento y CPU de verdad (multiprocessing); hilos para esperas, servicios de fondo y todo lo ligero que comparte memoria.

Entrevista de trabajo

Las preguntas que te harán

1

¿Qué es un hilo y en qué se diferencia de un proceso?

2

¿Cómo creas y lanzas varios hilos con argumentos y nombres?

3

¿Qué es un daemon?
¿Cuándo lo usarías?

4

¿Qué es el GIL y cómo afecta al rendimiento?

5

¿Cómo esperas a varios hilos a la vez?



Las reinas son la 1 y la 4. Para la 1: memoria, coste, comunicación y aislamiento (la tabla del bloque 01) y remata con la moraleja del ring. Para la 4: qué es el GIL, por qué no acelera CPU-bound, por qué sí I/O-bound, y cuándo toca multiprocessing.

No hay preguntas tontas



¿Un hilo puede crear otro hilo?

Sí, sin problema.



¿Cuántos puedo crear?

Miles como máximo (~1 MB de pila cada uno).



¿Puedo matar un hilo desde fuera?

No limpiamente: variable bandera que el hilo consulta.



¿sleep(0) sirve de algo?

Sí: cede la CPU voluntariamente.



¿Los hilos tienen prioridad?

En Python, no: decide el scheduler del SO.



¿Qué pasa si no llamo a join()?

El hilo corre igual; pierdes orden y sincronización.

Resumen de la unidad



Hilos

Memoria compartida, baratos, sin aislamiento



Ciclo básico

Thread · start() · join()



Daemon y Timer

Fondo prescindible; aviso diferido



GIL

CPU no acelera; I/O sí



Estados

NUEVO →
EJECUTABLE →
EJECUCIÓN ⇌
BLOQUEADO →
TERMINADO

